Práctica 1

Sergio López Esteve

Sistemas Operativos

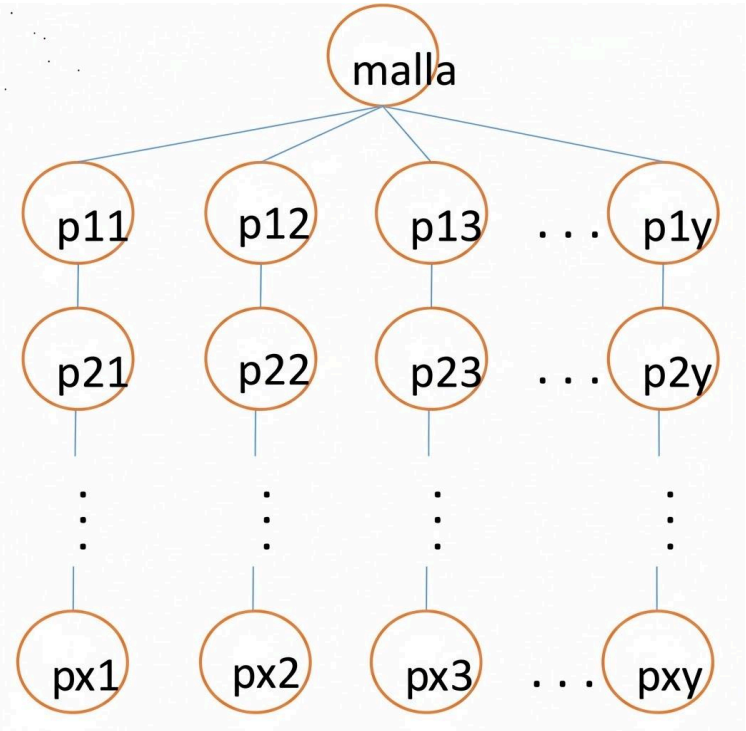
Índice

1a. Ejercicio 1a (malla.c)....3
1a.1. Enunciado y Estructura del Árbol.... 3
1a.2. Identificación y Justificación de Tareas....3
1a.3. Diagrama de Secuencia.... 4
1a.4. Análisis del Código Fuente.... 4
1a.4.1. Funciones de utilidad (parse(), despertar() y funcion_pstree())....5
1a.4.2. Proceso superpadre (main() y horizontal())....5
1a.4.3. Procesos de las columnas (vertical())....7
1b. Ejercicio 1b (ejec.c)....8
1b.1. Enunciado, Estructura del Árbol y Resultado Esperado....8
1b.2. Identificación y Justificación de Tareas....9
1b.3. Diagrama de Secuencia....9
1b.4. Análisis del Código Fuente....10
1b.4.1. Funciones de utilidad (parse(), manejador_despertar() y funcion_pstree())....10
1b.4.2. Proceso ejec (main()) y Proceso A....11
1b.4.3. Proceso intermedio B....12
1b.4.4. Procesos X, Y y Z....13
2. Ejercicio 2 (hacha.c)....15
2.1. Enunciado y explicación del ejercicio....15
2.2. Identificación y Justificación de Tareas....15
2.3. Diagrama de Secuencia....16
2.4. Análisis del Código Fuente....17
2.4.1. Gestión de argumentos y preparación del fichero (parse(), calcularNumeroTrozos() y abrirArchivo())....17
2.4.2. Creación de tuberías y procesos (main() y crearPipesyProcesos())....18
2.4.3. Proceso Padre: lectura y envío por tubería....19
2.4.4. Procesos Hijo: recepción por tubería y creación de trozos....20
3. Ejercicio 3 (hijos.c)....22
3.1. Enunciado, Estructura del Árbol y Resultado Esperado....22
3.2. Identificación y Justificación de Tareas....23
3.3. Diagrama de Secuencia....24
3.4. Análisis del Código Fuente....25
3.4.1. Argumentos y memoria compartida (parse(), creaComp() y borraComp())....25
3.4.2. Superpadre y print final (main() y printSuperPadre())....26
3.4.3. Generación árbol (crearProcesos() y crearUltimaFila())....27
3.4.4. Procesos hijo, registro en memoria compartida e impresión (codigoHijosFinales())....29

1a. Ejercicio 1a (*malla.c)*

1a.1. Enunciado y Estructura del Árbol

a) Realiza un programa llamado **malla.c** que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema "*pstree –c*" **(2 puntos)**

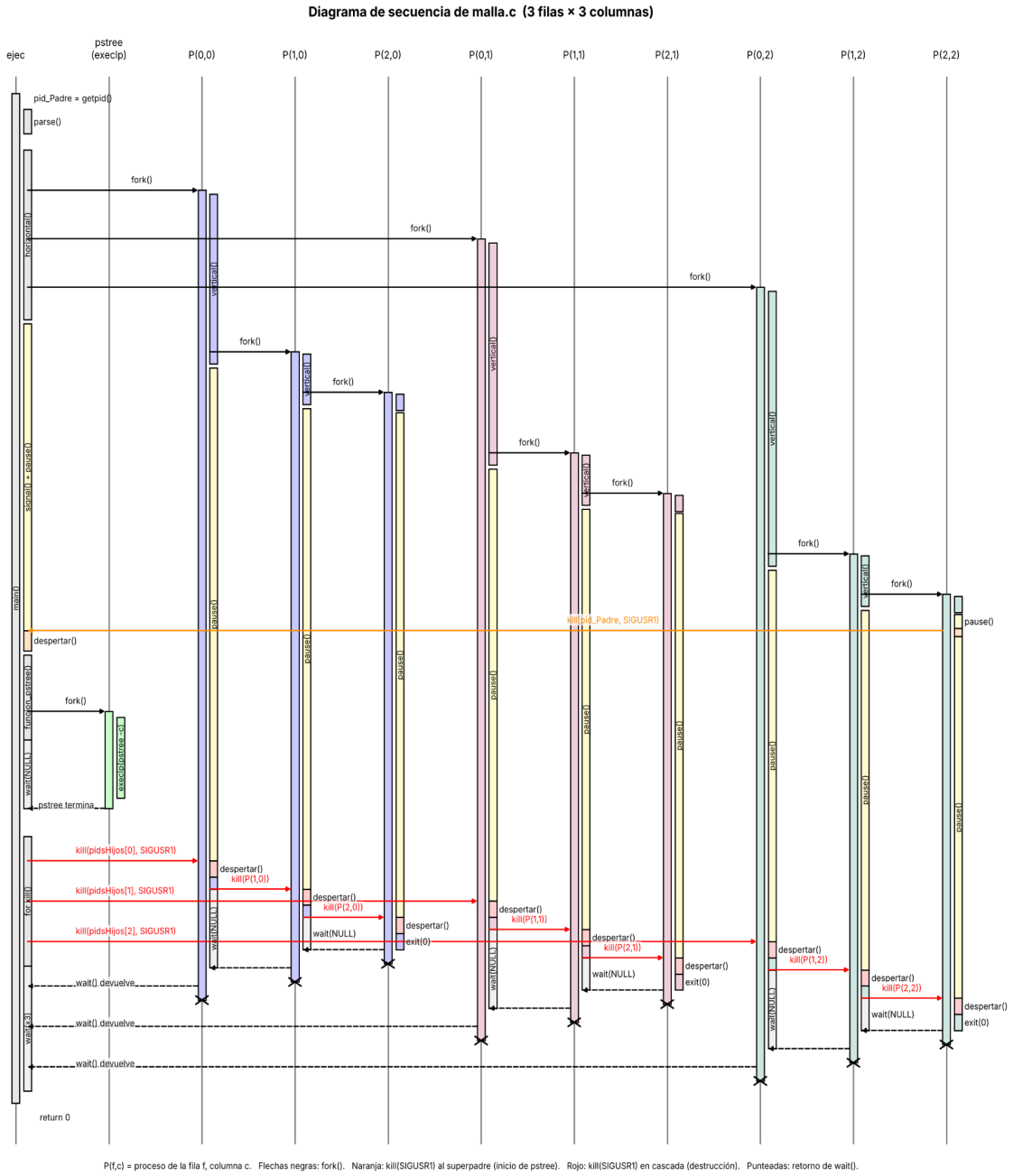


1a.2. Identificación y Justificación de Tareas

A partir del enunciado del ejercicio se identifican 3 fases claras: la creación del árbol de procesos, la comprobación con el pstree y la destrucción del árbol de forma controlada. La primera fase consistirá en que el proceso "malla" cree la primera fila de procesos de la malla iterando por cada columna a crear en la malla, y que estos primeros procesos se encarguen de crear hacia abajo sus respectivas columnas en un bucle de creación vertical (los padres saldrán del bucle en cada iteración y los hijos permanecerán). La segunda fase consistirá en localizar al último proceso de la malla (*pxy*), que este mande una señal al súper padre para que cree otro hijo que realice el pstree y muera. Y por último la tercera fase consistirá en que todos los procesos empezando por el súper padre manden señales en cascada (y que todos esperen con un wait, menos los últimos) hasta que se llegue a los últimos procesos y el árbol muera de abajo arriba para no dejar procesos huérfanos.

1a.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida de la malla para un caso 3x3: la creación por filas y columnas, el aviso del último proceso al superpadre para ejecutar pstree -c y la destrucción en cascada controlada.



1a.4. Análisis del Código Fuente

A continuación, se desglosa el propósito y funcionamiento de cada una de las funciones definidas en el código del programa:

1a.4.1. Funciones de utilidad (parse(), despertar() y funcion_pstree())

Función parse()

El propósito de esta función es validar los argumentos. Comprueba que el número de argumentos sea exactamente 3 (el nombre del programa más el número de filas y columnas); en caso contrario, informa del error por pantalla y finaliza la ejecución del programa.

```c
// GESTIÓN DE ARGUMENTOS
void parse(int argc, char *argv[]){
    if(argc != 3){
        printf("ERROR: número de argumentos incorrectos\n");
        exit(1);
    }
}
```

Función despertar()

Actúa como un manejador de señal vacío para SIGUSR1. Su única función es capturar la señal para interrumpir el pause() de los procesos y permitir que reanuden su ejecución.

```c
void despertar(int sig){} // MANEJADOR VACÍO
```

Función funcion_pstree()

Su objetivo es mostrar la estructura de la malla en consola. Para ello, crea un proceso hijo temporal con fork() que ejecuta el comando pstree -c mediante execlp().

Llamadas al Sistema Utilizadas: fork(), execlp(), exit().

```c
// PSTREE
void funcion_pstree(){
    if(fork()==0){
        // HIJO
        char strPidP[10];
        sprintf(strPidP, "%d", pid_Padre);
        execlp("pstree","pstree","-c", strPidP ,NULL);

        exit(0);
    }
}
```

1a.4.2. Proceso superpadre (main() y horizontal())

Función main()

Es el punto de entrada y la función principal del programa. Se encarga de validar y convertir los argumentos, almacenar su propio PID en la variable global pid_Padre y lanzar la creación de la malla mediante horizontal(). Después, se pausa con pause() hasta que el último hijo creado le avisa con SIGUSR1, y al despertar, ejecuta funcion_pstree() y espera a que termine. Finalmente inicia la destrucción controlada enviando SIGUSR1 a toda la primera fila y esperando a cada columna con wait(NULL).

Llamadas al Sistema Utilizadas: getpid(), signal(), pause(), wait(), kill().

```c
int main(int argc, char *argv[]){
    // ARGUMENTOS
    parse(argc, argv);
    int numFila = atoi(argv[1]);
    int numCol = atoi(argv[2]);

    pid_Padre = getpid(); // Guardamos el pid del padre
    pid_t pidsHijos[numCol]; // y declaramos un array para el de los hijos

    horizontal(numFila, numCol, pidsHijos);

    signal(SIGUSR1, despertar); // Configurar señal
    pause(); // Espera la señal que le manda el último hijo

    funcion_pstree();
    wait(NULL); // Espera a que pstree termine
    for(int i=0; i<numCol; i++){ // Manda la señal de terminación a todos los hijos que tiene
         kill(pidsHijos[i], SIGUSR1);
    }


    // Espera por todos los hijos que tenga
    for (int i = 0; i < numCol; i++) {
        wait(NULL);
    }

    return 0;
}
```

Función horizontal()

El propósito de esta función es generar la primera fila de procesos a partir del superpadre. Mediante un bucle de numCol iteraciones realiza llamadas a fork(): el superpadre guarda el PID de cada hijo en el vector pidsHijos[] para poder enviarles la señal de destrucción al final, mientras que cada hijo continúa su ejecución en la función vertical() para crear su propia columna.

Llamadas al Sistema Utilizadas: fork().

```c
// CREA LA PRIMERA FILA DE PROCESOS
void horizontal(int numFila, int numCol, pid_t pidsHijos[]){
    for(int c=0; c<numCol; c++){

        pid_t pidAux = fork();
        if(pidAux == 0){  // Si es un hijo crea su columna, si no es el padre y sigue creando la primera fila.
            vertical(c, numFila, numCol);
        }else{
            pidsHijos[c] = pidAux; // Guarda los pids de los hijos para la destrucción del arbol
        }
    }
}
```

1a.4.3. Procesos de las columnas (vertical())

Función vertical()

Define el comportamiento de todos los procesos hijos para construir las columnas y destruirlas en cascada. Primero ejecuta un bucle desde la fila 1 hasta numFila - 1 haciendo fork(), donde cada padre sale del bucle con break tras crear a su único hijo directo y el hijo continúa iterando hacia abajo. Una vez creados, todos configuran SIGUSR1, y si el proceso es el último de la última fila y columna, avisa al superpadre con kill(pid_Padre, SIGUSR1) para que lance pstree. Finalmente, todos esperan en pause() la señal de destrucción. Al despertar, los procesos intermedios (pidHijo > 0) propagan SIGUSR1 a su hijo inferior y esperan su muerte con wait(NULL) antes de terminar con exit(0), mientras que los últimos realizan el exit(0) directamente y se realiza la muerte de forma ascendente.

Llamadas al Sistema Utilizadas: fork(), signal(), kill(), pause(), wait(), exit().

```c
// CREA LAS COLUMNAS DE HIJOS
void vertical(int columnaActual,int numFila, int numCol){
    int filaActual;
    pid_t pidHijo = 0;

    // CREACIÓN COLUMNAS (f=1 porque la primera fila ya la ha creado el superpadre)
    for(filaActual=1; filaActual<numFila; filaActual++){
        pidHijo = fork(); // Se guarda el pid del hijo
        if(pidHijo > 0){ // Si es el padre se guarda el numero de fila y sale del bucle
            break;
        }
    }
    // SEÑALES
    signal(SIGUSR1, despertar);

    // Último hijo
    if(filaActual == numFila && columnaActual == (numCol-1)){
            kill(pid_Padre, SIGUSR1); // Manda la señal al padre para hacer pstree
    }

    pause(); // Todos los hijos esperan a la señal en cascada del superpadre

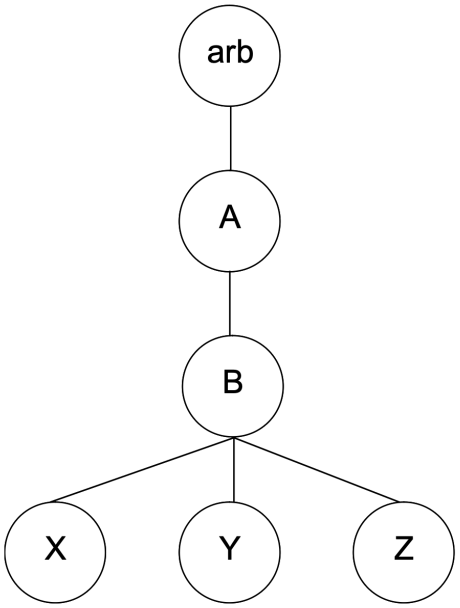

    if(pidHijo > 0){ // Si no es el último de la columna manda el kill hacia abajo y espera
        kill(pidHijo, SIGUSR1);
        wait(NULL);
    }

    exit(0);
}
```

1b. Ejercicio 1b (*ejec.c)*

1b.1. Enunciado, Estructura del Árbol y Resultado Esperado

b) Realiza un programa llamado **ejec.c** que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos). **(2 puntos)**



El programa debe mostrar por la salida estándar la siguiente información:

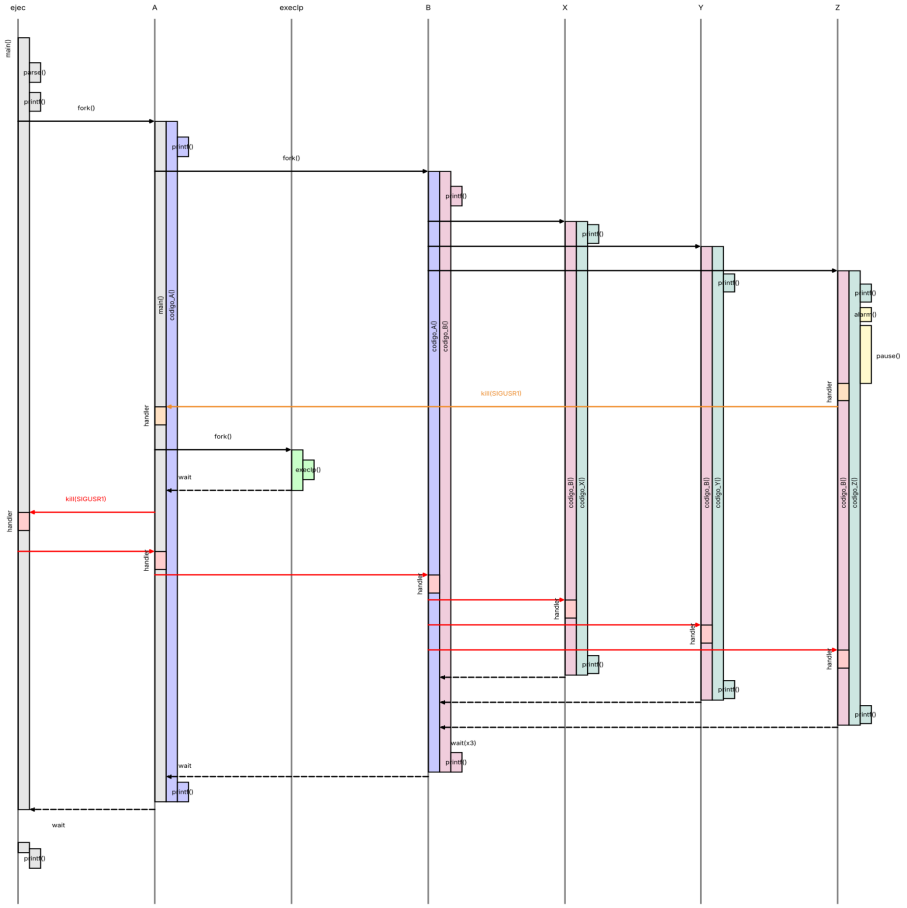
```
$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
/*Tras un intervalo de 15 segundos aparecerá*/
/*Resultado del comando "pstree" */
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

1b.2. Identificación y Justificación de Tareas

A partir del enunciado del ejercicio se identifican 3 fases claras: la creación del árbol de procesos, el uso de señales con alarma en el proceso Z y con el pstree en A, y la destrucción del árbol de forma controlada. La primera fase consistirá en que el proceso "ejec" cree al proceso A, este cree al proceso B, y B se encargue de crear mediante un bucle de 3 iteraciones a sus hijos X, Y y Z, imprimiendo cada proceso su PID y el de sus padres al nacer y quedándose todos (menos el Z) en espera con un pause. La segunda fase consistirá en que el proceso Z programe una alarma con los segundos recibidos por argumento (para no usar la llamada prohibida sleep) y, cuando despierte, que Z le mande una señal al proceso A para que cree un hijo temporal que ejecute el pstree. Y por último la tercera fase consistirá en que el proceso A, tras terminar el pstree, mande una señal de aviso al súper padre (ejec) para que este inicie la cascada de señales de destrucción hacia abajo (de ejec a A, de A a B y de B a X, Y y Z), esperando cada padre con un wait a que sus hijos terminen para que el árbol muera de abajo arriba sin dejar procesos huérfanos.

1b.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida del árbol de procesos: su creación escalonada, la espera mediante alarm() en el proceso Z, el aviso al proceso A para ejecutar pstree -c y la destrucción en cascada controlada desde el superpadre hasta los últimos hijos.



1b.4. Análisis del Código Fuente

A continuación, se desglosa el propósito y funcionamiento de cada una de las funciones definidas en el código del programa:

1b.4.1. Funciones de utilidad (parse(), manejador_despertar() y funcion_pstree())

Función parse()

El propósito de esta función es validar los argumentos. Comprueba que el número de argumentos sea exactamente 2 (el nombre del programa más los segundos de la alarma), y en caso contrario, muestra un mensaje de error y finaliza la ejecución del programa.

```
void parse(int argc, char *argv[]){
    if(argc != 2){
        printf("ERROR: número de argumentos incorrectos\n");
        exit(1);
    }
}
```

Función manejador_despertar()

Actúa como un manejador de señal vacío tanto para SIGUSR1 como para SIGALRM. Su única función es capturar la señal para interrumpir el pause() de los procesos y permitir que continúen su ejecución.

```
void manejador_despertar(int sig){}
```

Función funcion_pstree()

Su objetivo es mostrar la estructura del árbol en consola. Para ello, crea un proceso hijo temporal con fork() que ejecuta el comando pstree -c mediante execlp().

Llamadas al Sistema Utilizadas: fork(), execlp(), exit().

```
void funcion_pstree(){
    if(fork()==0){
        // HIJO
        char strPidP[10];
        sprintf(strPidP, "%d", pid_Padre);
        execlp("pstree","pstree","-c", strPidP ,NULL);

        exit(0);
    }
}
```

1b.4.2. Proceso ejec (main()) y Proceso A

Función main()

Es el punto de entrada y la función principal del programa. Se encarga de validar y convertir el argumento de los segundos, guardar su propio PID en pid_Padre, imprimir su mensaje de inicio y crear al proceso A mediante fork(). Después, el superpadre se pausa con pause() hasta que el proceso A le avisa con SIGUSR1 de que ya se ha hecho el pstree. Al despertar, inicia la destrucción controlada enviando SIGUSR1 a A, esperando a que muera con wait(NULL) e imprimiendo su propio mensaje de muerte.

Llamadas al Sistema Utilizadas: getpid(), fork(), signal(), pause(), kill(), wait(), exit().

```c
int main(int argc, char *argv[]){

    parse(argc, argv);
    int segundos_alarma = atoi(argv[1]);

    pid_Padre = getpid();
    printf("Soy el proceso ejec: mi pid es %d\n", pid_Padre);

    switch(fork()){
        case -1:
            // ERROR
            perror("Error al crear el proceso A\n");

            exit(1);
        case 0:
            // HIJO
            pid_A = getpid();
            codigo_A(segundos_alarma);

        default:
            // PADRE
            signal(SIGUSR1, manejador_despertar);
            pause();
            // Recibe señal de despertar de A

            kill(pid_A,SIGUSR1); // Manda señal a A
            wait(NULL);

            printf("Soy ejec (%d) y muero\n", getpid());
            exit(0);
    }
    return 0;
}
```

Función codigo_A()

Define el comportamiento del proceso A. Primero imprime su PID y el de su padre, crea al proceso B con fork() y se queda en pause(), esperando la señal del proceso Z. Cuando Z lo despierta con SIGUSR1, ejecuta funcion_pstree() y espera a que termine con wait(NULL). Acto seguido, manda una señal con kill(pid_Padre, SIGUSR1) para avisar al superpadre y vuelve a hacer pause(). Cuando el superpadre le devuelve la señal de destrucción, A se la propaga a B con kill(), espera a que B termine con wait(NULL), imprime su muerte y termina con exit(0).

Llamadas al Sistema Utilizadas: getpid(), fork(), signal(), pause(), wait(), kill(), exit().

```c
void codigo_A(int segundos){
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d.\n", getpid(), pid_Padre);

    pid_t pid_B = fork();

    switch(pid_B){
        case -1:
            // ERROR
            printf("Error al crear el proceso B.\n");

            exit(1);
        case 0:
            // HIJO
            codigo_B(segundos);

        default:
            // PADRE
            signal(SIGUSR1, manejador_despertar);
            pause();

            funcion_pstree();
            wait(NULL);

            kill(pid_Padre, SIGUSR1); // Señal que le manda al superpadre
            pause();
            // Recibe del superpadre

            kill(pid_B, SIGUSR1); // Manda señal a B
            wait(NULL);

            printf("Soy A (%d) y muero\n", getpid());
            exit(0);
    }
}
```

1b.4.3. Proceso intermedio B

Función codigo_B()

Se encarga de imprimir los datos del proceso B y crear a los 3 procesos finales mediante un bucle de 3 iteraciones con fork(), guardando sus PIDs en el array pids_xyz[] y enviando a cada hijo a su función correspondiente (codigo_X, codigo_Y y codigo_Z). Una vez creados, B se bloquea con pause() esperando la señal de destrucción de A. Al recibirla, recorre el array pids_xyz[] mandando SIGUSR1 con kill() a sus 3 hijos, los espera con un bucle de 3 wait(NULL), imprime su mensaje de muerte y finaliza con exit(0).

Llamadas al Sistema Utilizadas: getpid(), fork(), signal(), pause(), kill(), wait(), exit().

```c
void codigo_B(int segundos){
    printf("Soy el proceso B mi pid es %d. Mi padre es %d. Mi abuelo es %d.\n", getpid(), pid_A, pid_Padre);

    pid_t pids_xyz[3];

    for(int i=0; i<3; i++){
        pid_t pid_aux = fork();

        switch(pid_aux){
            case -1:
                printf("Error al crear un hijo de B.");

                exit(1);
            case 0:
                if(i==0) codigo_X();
                if(i==1) codigo_Y();
                if(i==2) codigo_Z(segundos);
            default:
                pids_xyz[i] = pid_aux;
                break;
        }
    }
    signal(SIGUSR1, manejador_despertar);
    pause(); // Espera la señal de A

    // Propaga la señal a sus 3 hijos
    for(int i=0; i<3; i++){
        kill(pids_xyz[i], SIGUSR1);
    }

    // Espera a que los 3 hijos mueran
    for(int i=0; i<3; i++){
        wait(NULL);
    }

    printf("Soy B (%d) y muero.\n", getpid());
    exit(0);
}
```

1b.4.4. Procesos X, Y y Z

Funciones codigo_X() y codigo_Y()

Definen el comportamiento de los procesos X e Y. Ambos imprimen al nacer su PID junto con el de su padre (B), abuelo (A) y bisabuelo (ejec), configuran la señal SIGUSR1 y se quedan esperando en pause(). Cuando reciben la señal de destrucción por parte de B, imprimen su mensaje de muerte y terminan directamente con exit(0).

Llamadas al Sistema Utilizadas: getpid(), getppid(), signal(), pause(), exit().

```c
void codigo_Y(){
    printf("Soy el proceso Y mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d.\n", getpid(), getppid(), pid_A, pid_Padre);
    signal(SIGUSR1, manejador_despertar);
    pause();

    printf("Soy Y (%d) y muero\n", getpid());
    exit(0);
}

void codigo_X(){
    printf("Soy el proceso X mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d.\n", getpid(), getppid(), pid_A, pid_Padre);
    signal(SIGUSR1, manejador_despertar);
    pause();

    printf("Soy X (%d) y muero\n", getpid());
    exit(0);
}
```

Función codigo_Z()

Además de imprimir su línea de padres como sus hermanos, el proceso Z es el encargado de llevar la temporización del programa. Configura la señal SIGALRM, programa la cuenta atrás con alarm(segundos) y se espera con pause() (sin usar sleep). Cuando salta la alarma y se despierta, envía la señal SIGUSR1 al proceso A mediante kill(pid_A, SIGUSR1) para que ejecute el pstree. Por último, configura SIGUSR1 y vuelve a quedarse en pause() hasta recibir la orden de muerte de B, momento en el que imprime su mensaje final y hace exit(0).

Llamadas al Sistema Utilizadas: getpid(), getppid(), signal(), alarm(), pause(), kill(), exit().

```c
void codigo_Z(int segundos){
    printf("Soy el proceso Z mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d.\n", getpid(), getppid(), pid_A, pid_Padre);

    signal(SIGALRM, manejador_despertar);
    alarm(segundos);
    pause();

    kill(pid_A, SIGUSR1);

    signal(SIGUSR1, manejador_despertar);
    pause();

    printf("Soy Z (%d) y muero\n", getpid());
    exit(0);
}
```

2. Ejercicio 2 (*hacha.c)*

2.1. Enunciado y explicación del ejercicio

Realizar un programa llamado **hacha.c** que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, … . Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <archivo> <tamaño>
<archivo> nombre del archivo a dividir
<tamaño> tamaño en bytes de los archivos divididos

- Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

Consideraciones

- Las entradas y salidas a los archivos se realizarán con llamadas al sistema estudiadas en la asignatura (no se puede utilizar printf, scanf, etc.)
- El padre creará tantos hijos como fragmentos del archivo a realizar, siendo decisión del alumno si los hijos se lanzan secuencial o concurrentemente.

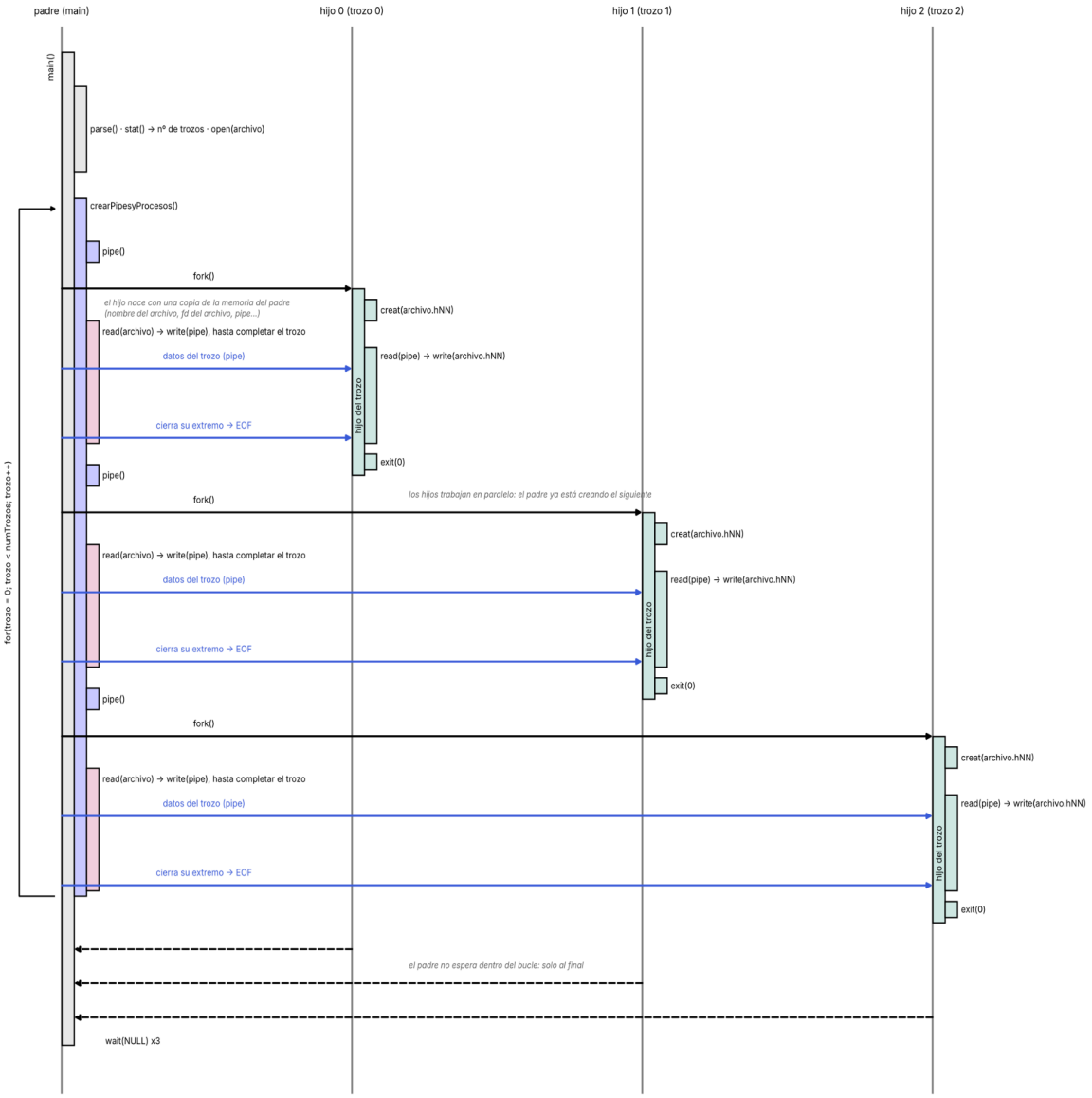
2.2. Identificación y Justificación de Tareas

A partir del enunciado del ejercicio se identifican 3 fases claras: la preparación de todo lo necesario para comenzar la creación de pipes y procesos, precisamente la creación de estas tuberías y procesos hijos por cada fragmento, y la lectura y envío de bytes del padre a los hijos hasta completar todos los trozos. La primera fase consistirá en la gestion de los argumentos, en obtener el tamaño total del archivo original mediante la llamada stat para dividirlo entre el tamaño de trozo indicado por argumento (añadiendo un trozo más si el resto no es exacto) y así hallar la cantidad de trozos a realizar, y abrir el archivo una única vez en modo lectura (para así obtener su descriptor) con open. La segunda fase consistirá en realizar un bucle por cada trozo en el que el padre cree una tubería (pipe) y un hijo (fork), cerrando el extremo de lectura en el padre y el de escritura en el hijo. Y por último la tercera fase

consistirá en que el padre lea con read los bytes correspondientes a ese trozo del archivo original y se los mande con write por la tubería al hijo (el cual habrá creado el archivo .h00, .h01... con creat para ir escribiendo en él), cerrando el padre su descriptor de escritura al terminar el trozo para que el hijo detecte el fin de archivo (EOF), cierre sus descriptores y muera, esperando finalmente el padre a todos los hijos con wait.

2.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida del programa para un caso de 3 trozos: el cálculo inicial con stat() y open(), el bucle en el que se crean cada tubería e hijo, el envío de datos mediante read() y write() hasta cerrar el extremo de la tubería (EOF), y la espera final del padre con wait(NULL).



2.4. Análisis del Código Fuente

A continuación, se desglosa el propósito y funcionamiento de cada una de las funciones definidas en el código del programa:

2.4.1. Gestión de argumentos y preparación del fichero (parse(), calcularNumeroTrozos() y abrirArchivo())

Función parse()

El propósito de esta función es validar los argumentos. Comprueba que el número de argumentos sea exactamente 3 (el nombre del programa, el nombre del archivo y el tamaño de cada trozo), y en caso contrario, muestra un mensaje de error y finaliza la ejecución del programa.

```c
// ANALISIS ARGUMENTOS
void parse(int argc, char *argv[]){
    if(argc != 3){
        printf("ERROR: número de argumentos incorrectos.\n");
        exit(1);
    }
}
```

Función calcularNumeroTrozos()

Se encarga de calcular en cuántos fragmentos se va a dividir el archivo. Primero comprueba que el tamaño de trozo sea mayor que 0 y después utiliza la llamada al sistema stat() para obtener el tamaño total del archivo en bytes (st_size). Finalmente divide el tamaño total entre el tamaño de trozo, sumando un trozo adicional si la división no es exacta para no perder los bytes restantes.

Llamadas al Sistema Utilizadas: stat(), exit().

```c
int calcularNumeroTrozos(const char *nombreArchivo, int tamTrozo){
    struct stat statsArchivo;
    int tamArchivo, numTrozos;

    if(tamTrozo <= 0){
        perror("Error: tamaño de trozos inferior o igual a 0.");
        exit(1);
    }

    if(stat(nombreArchivo, &statsArchivo) < 0){

        perror("Error al calcular el tamaño del archivo.(stat)\n");
        exit(1);
    }else{
        tamArchivo = statsArchivo.st_size;
        numTrozos = tamArchivo/tamTrozo;
        if(tamArchivo%tamTrozo != 0){ // Si la división no es exacta se añade un trozo más
            numTrozos++;              // para leer lo restante.
        }
    }
    return numTrozos;
}
```

Función abrirArchivo()

Su objetivo es abrir el archivo original que se quiere trocear en modo solo lectura (O_RDONLY) mediante open() y devolver su descriptor de archivo, controlando si se produce algún error.

Llamadas al Sistema Utilizadas: open(), exit().

```c
int abrirArchivo(const char *nombreArchivo){

    int descriptorArchivo = open(nombreArchivo, O_RDONLY);
    if(descriptorArchivo < 0){
        perror("Error al abrir el archivo.\n");
        exit(1);
    }

    return descriptorArchivo;
}
```

2.4.2. Creación de tuberías y procesos (main() y crearPipesyProcesos())

Función main()

Es el punto de entrada y la función principal del programa. Se encarga de validar los argumentos, calcular el número de trozos con calcularNumeroTrozos(), abrir el fichero origen con abrirArchivo() y llamar a crearPipesyProcesos(). Una vez creados y enviados todos los trozos, cierra el descriptor del archivo original con close() y espera a que terminen todos los procesos hijos con un bucle de wait(NULL).

Llamadas al Sistema Utilizadas: close(), wait().

```c
int main(int argc, char *argv[]){

    parse(argc, argv); // Analisis argumentos

    char *nombreArchivo = argv[1]; // Nombre del archivo
    int tamTrozo = atoi(argv[2]); // Tamaño de los trozos en bytes

    int numTrozos = calcularNumeroTrozos(nombreArchivo, tamTrozo); // Calculamos numero de trozos
    int descriptorArchivo = abrirArchivo(nombreArchivo); // Abrimos y guardamos el descriptor

    crearPipesyProcesos(nombreArchivo, tamTrozo, numTrozos, descriptorArchivo);

    close(descriptorArchivo); // Se cierra el archivo

    // Se espera a todos los hijos
    for(int i=0; i < numTrozos; i++){
        wait(NULL);
    }
    return 0;
}
```

Función crearPipesyProcesos()

Recorre un bucle tantas veces como trozos haya que crear. En cada iteración crea una tubería nueva con pipe() y un proceso hijo con fork(). Para que la comunicación vaya del padre al hijo, el proceso hijo cierra el descriptor de escritura de la tubería con close(descriptoresPipe[1]) y pasa a ejecutar codigoHijo(), mientras que el padre cierra el descriptor de lectura con close(descriptoresPipe[0]) y ejecuta codigoPadre().

Llamadas al Sistema Utilizadas: pipe(), fork(), close(), exit().

```c
void crearPipesyProcesos(const char *nombreArchivo, int tamTrozo, int numTrozos, int descriptorArchivo){
    int descriptoresPipe[2];


    for(int trozo = 0; trozo < numTrozos; trozo++){
        // Crear tubería
        if(pipe(descriptoresPipe) < 0){
            perror("Error al crear la tubería");
            exit(1);
        }

        // Crear proceso
        pid_t pid = fork();

        switch(pid){
            case -1:
                // ERROR
                perror("Error al crear un hijo");
                exit(1);
            case 0:
                // HIJO
                close(descriptoresPipe[1]);
                codigoHijo(descriptoresPipe[0], nombreArchivo, trozo);
            default:
                // PADRE
                close(descriptoresPipe[0]);
                codigoPadre(descriptoresPipe[1], descriptorArchivo, tamTrozo);
        }
    }
}
```

2.4.3. Proceso Padre: lectura y envío por tubería

Función codigoPadre()

Se encarga de leer un trozo del archivo original y enviárselo al hijo correspondiente a través de la tubería. Como el descriptor del archivo original se mantiene abierto, el puntero de lectura avanza solo entre un trozo y el siguiente. Mediante un bucle while controla los bytes pendientes de ese trozo: lee del archivo con read() usando un buffer de 1024 bytes (ajustando la cantidad a leer si quedan menos de 1024 bytes pendientes para no pasarse al siguiente trozo) y los escribe en la tubería con write(). Al terminar de enviar el trozo, cierra el descriptor de escritura con close(descEscribirPipe) para que el hijo reciba la señal de fin de archivo (EOF).

Llamadas al Sistema Utilizadas: read(), write(), close(), exit().

```c
void codigoPadre(int descEscribirPipe, int descArchivo, int tamTrozo){

    char buffer[1024];
    int pendientes = tamTrozo;   // bytes que aún faltan por enviar de este trozo
    int aLeer, leidos;

    while(pendientes > 0){
        // Leemos como máximo lo que cabe en el buffer, sin pasarnos del trozo
        aLeer = sizeof(buffer);
        if(pendientes < aLeer){ // Si quedan menos bytes que el buffer,
            aLeer = pendientes; // leemos solo los pendientes para no pasarnos del trozo.
        }

        leidos = read(descArchivo, buffer, aLeer); // Bytes leidos
        if(leidos < 0){
            perror("Error al leer el archivo origen");
            exit(1);
        }

        if(leidos == 0){   // Si no hay más datos que enviar, fin de archivo.
            break;
        }

        // si no coinciden con los leidos y los escritos, error.
        if(write(descEscribirPipe, buffer, leidos) != leidos){
            perror("Error al escribir en la tubería");
            exit(1);
        }
        pendientes -= leidos;
    }

    close(descEscribirPipe);   // El hijo recibe EOF
}
```

2.4.4. Procesos Hijo: recepción por tubería y creación de trozos

Función codigoHijo()

Define el comportamiento de cada hijo para crear su archivo de trozo. Primero genera el nombre del nuevo archivo con extensión .h00, .h01, etc. usando sprintf() y crea el fichero con creat(). Después entra en un bucle donde lee los datos que le llegan por la tubería con read() y los escribe en el archivo nuevo con write() hasta que read() devuelve 0 (cuando el padre cierra la tubería y llega el EOF). Por último, cierra los descriptores tanto del archivo creado como de la tubería con close() y muere con exit(0).

Llamadas al Sistema Utilizadas: creat(), read(), write(), close(), exit().

```c
void codigoHijo(int descLeerPipe, const char *nombreArchivo, int trozo){

    char nombreDestino[512]; // 512 para cubrir todos los posibles nombres de archivo
    char buffer[1024];
    int descDestino, leidos;

    // Nombres: archivo.h00, archivo.h01, ...
    sprintf(nombreDestino, "%s.h%02d", nombreArchivo, trozo);

    // Creamos el archivo destino
    descDestino = creat(nombreDestino, 0666);
    if(descDestino < 0){
        perror("Error al crear el archivo del trozo");
        exit(1);
    }

    // Copiamos de la tubería al archivo hasta recibir EOF
    while((leidos = read(descLeerPipe, buffer, sizeof(buffer))) > 0){
        if(write(descDestino, buffer, leidos) != leidos){ // Si lo leído y lo escrito no
            perror("Error al escribir el trozo");         // coinciden, error.
            exit(1);
        }
    }

    if(leidos < 0){
        perror("Error al leer de la tubería");
        exit(1);
    }

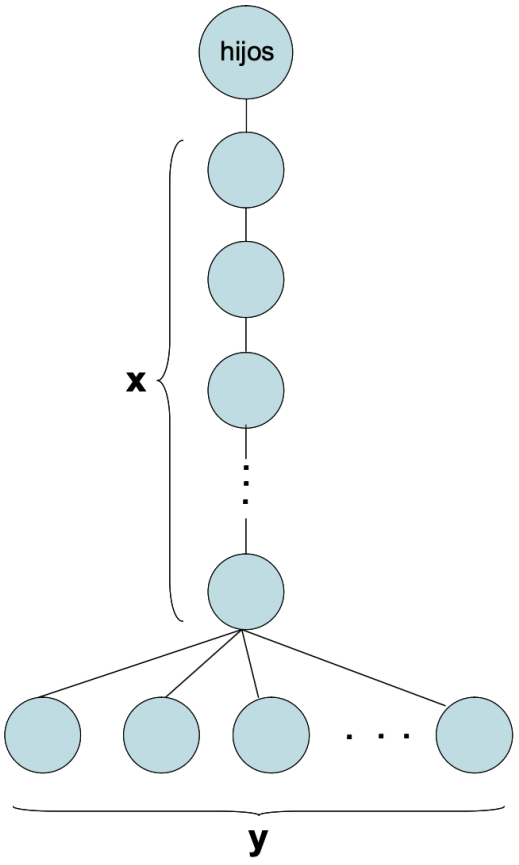
    close(descDestino);  // Al terminar de copiar se cierran ambos descriptores
    close(descLeerPipe); // para no ocupar espacios en la tabla.
    exit(0); // Muere el hijo
}
```

3. Ejercicio 3 (*hijos.c)*

3.1. Enunciado, Estructura del Árbol y Resultado Esperado

Diseña un programa llamado **hijos.c** que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros:

$hijos x y



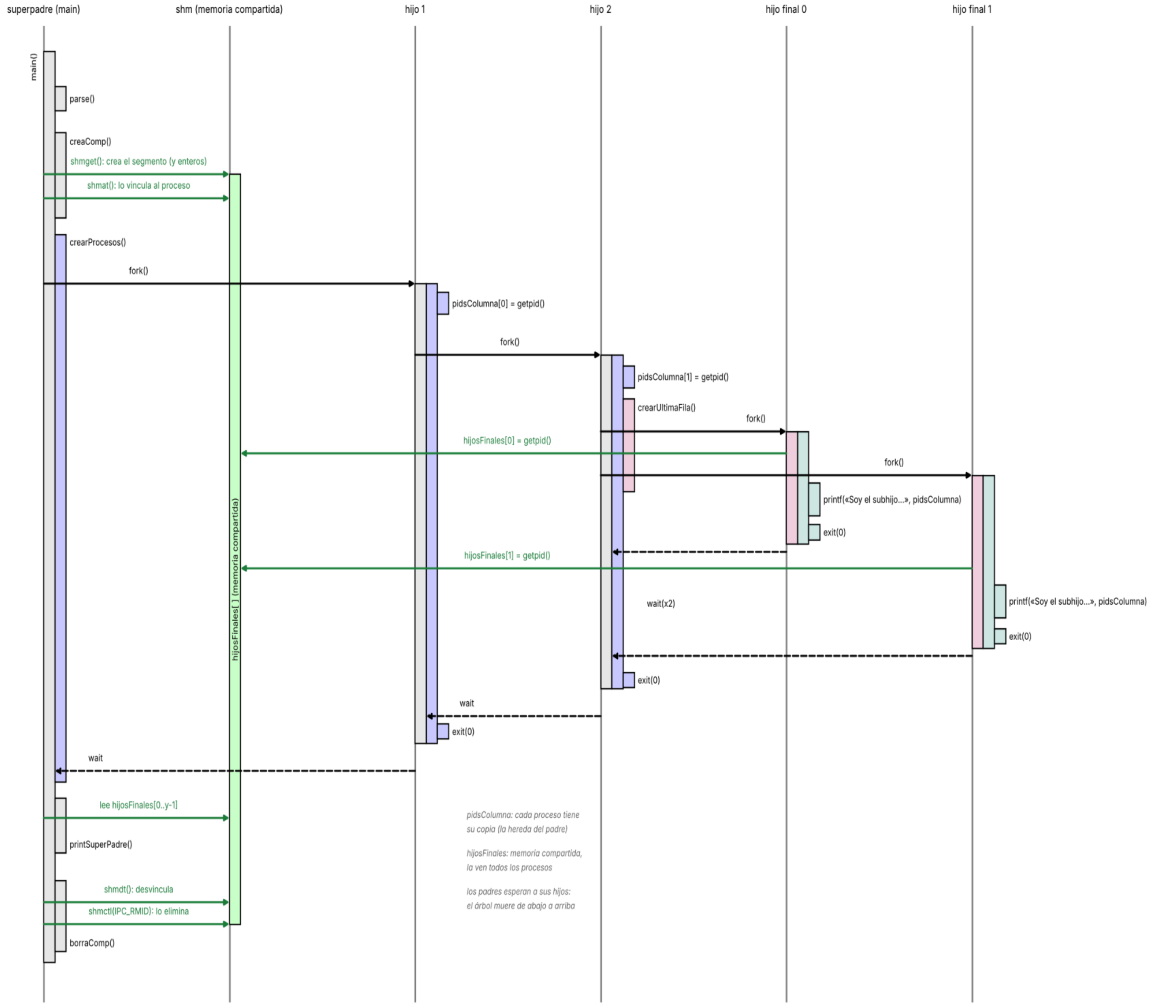
El proceso **hijos** debe mostrar el siguiente mensaje:
"Soy el superpadre (pid) : mis hijos finales son: `pidx1, pidx2, pidx3, …, pidxy`"
Los procesos de la parte baja deben mostrar el siguiente mensaje**:**
"Soy el subhijo pidx1, mi padres son: `pid1, pid2, …, pidx`"

3.2. Identificación y Justificación de Tareas

A partir del enunciado del ejercicio se identifican 3 fases claras: crear el segmento de memoria compartida y vincular este al proceso padre, crear el árbol de procesos e imprimir los PIDs, y por último, vincular y liberar la memoria compartida. Además hay que tener en cuenta que para la primera impresión necesitamos memoria compartida obligatoriamente, pero que en el caso de la segunda impresión no es necesario ya que los hijos pueden heredar el array de PIDs de sus padres e imprimirlo. Dicho esto, la primera fase consistirá en validar que x e y sean mayores a 0 y que el superpadre cree con shmget y vincule con shmat un segmento de memoria compartida (en este caso un array global hijosFinales) donde cabrán los PIDs de los y hijos finales. La segunda fase consistirá en crear una columna vertical de x procesos mediante un bucle de fork en el que cada hijo guarda su PID en el array local pidsColumna[] y cada padre espera con wait a que termine su hijo, encargándose el último proceso de la columna de hacer otro bucle de y iteraciones con fork para crear a los hijos finales. Cada subhijo deberá guardar su propio PID en el array de memoria compartida (hijosFinales[i]) y deberá imprimir todos sus padres leyendo el array heredado pidsColumna[], de modo que cuando los subhijos mueran y los wait suban en cascada hasta el superpadre, este imprima los PIDs guardados en hijosFinales[]. Por último, el padre también se encargará de desvincular y borrar la memoria compartida con shmdt y shmctl.

3.3. Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida del programa: la reserva de memoria compartida en el superpadre, la creación de la columna vertical acumulando los PIDs en pidsColumna[], el registro de los subhijos finales en la memoria compartida y la impresión y borrado del segmento al volver al superpadre.



3.4. Análisis del Código Fuente

A continuación, se desglosa el propósito y funcionamiento de cada una de las funciones definidas en el código del programa:

3.4.1. Argumentos y memoria compartida (parse(), creaComp() y borraComp())

Función parse()

El propósito de esta función es validar los argumentos. Comprueba que se reciban exactamente 3 argumentos (el ejecutable más x e y) y además comprueba que tanto x como y sean mayores que 0; en caso contrario, muestra error y termina el programa.

```c
// ANALISIS ARGUMENTOS
void parse(int argc, char *argv[]){
    if(argc != 3){
        printf("ERROR: número de argumentos incorrectos.\n");
        exit(1);
    }if(atoi(argv[1])<=0 || atoi(argv[2])<=0){
        printf("ERROR: las dimensiones del arbol deben ser mayores a 0.\n");
        exit(1);
    }
}
```

Función creaComp()

Se encarga de inicializar la memoria compartida antes de crear ningún hijo. Reserva un segmento de tamaño y * sizeof(int)(la cantidad de hijos finales por el tamaño de un entero) mediante shmget() y después lo vincula al proceso con shmat() en el puntero global hijosFinales, para que todos los procesos lo hereden al hacer fork().

Llamadas al Sistema Utilizadas: shmget(), shmat(), exit().

```c
void creaComp(int y){
    // Crear segmento
    shmid = shmget(IPC_PRIVATE, y * sizeof(int), IPC_CREAT | 0666); // Creamos el segmento compartido de tamaño
    if(shmid == -1){                                                // enteros * cantidad de hijos finales en bytes
        perror("Error al crear la memoria compartida");
        exit(1);
    }

    // Vincular segmento al proceso
    hijosFinales = (int *) shmat(shmid, 0, 0);
    if(hijosFinales == (int *) -1){
        perror("Error al vincular la memoria compartida");
        exit(1);
    }
}
```

Función borraComp()

Su función es liberar los recursos de memoria compartida al terminar el programa. Primero desvincula el puntero hijosFinales del proceso usando shmdt() y después elimina el segmento del sistema con shmctl(shmid, IPC_RMID, 0).

Llamadas al Sistema Utilizadas: shmdt(), shmctl(), exit().

```c
void borraComp(void){
    // Desvincular segmento
    if(shmdt((char *) hijosFinales) < 0){
        perror("Error al desvincular la memoria compartida");
        exit(1);
    }

    // Eliminar segmento
    if(shmctl(shmid, IPC_RMID, 0) < 0){
        perror("Error al borrar la memoria compartida");
        exit(1);
    }

    hijosFinales = NULL;
}
```

3.4.2. Superpadre y print final (main() y printSuperPadre())

Función main()

Es el punto de entrada y la función principal del programa. Comprueba y convierte los argumentos x e y a enteros, llama a creaComp() para preparar la memoria compartida y ejecuta crearProcesos(). Cuando toda la rama de procesos termina y el flujo vuelve al superpadre, llama a printSuperPadre() para mostrar los subhijos finales y elimina la memoria compartida con borraComp().

```c
int main(int argc, char *argv[]){

    parse(argc, argv);
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);

    creaComp(y); // Crea el segmento de memoria compartida y lo vincula con el proceso

    crearProcesos(x, y); // Crea el arbol y hace los prints
    printSuperPadre(y);

    borraComp(); // Desvincula el segmento del proceso y lo elimina

    return 0;
}
```

Función printSuperPadre()

Imprime el PID del superpadre y recorre con un bucle las *y* posiciones del array de memoria compartida hijosFinales[] para mostrar por pantalla los PIDs que han ido dejando escritos los subhijos de la última fila.

Llamadas al Sistema Utilizadas: getpid().

```c
void printSuperPadre(int y){
    printf("Soy el superpadre (%d) : mis hijos finales son: ", getpid());
    // imprimimos el pid de todos los hijos finales
    for(int i = 0; i < y; i++){
        printf("%d", hijosFinales[i]);
        if(i < y - 1){          // si no es el último, ponemos coma
            printf(", ");
        }
    }
    printf("\n");
}
```

3.4.3. Generación árbol (crearProcesos() y crearUltimaFila())

Función crearProcesos()

Se encarga de construir la columna vertical de x procesos y controlar su finalización. Declara el array pidsColumna[x] y hace un bucle de x iteraciones con fork(): en cada vuelta, el hijo guarda su PID en pidsColumna[i] y sigue en el bucle para crear al siguiente nivel, mientras que el padre se queda esperando con wait(NULL) (los padres intermedios hacen exit(0) cuando su hijo muere, y el superpadre en i == 0 hace return para volver al main). Cuando el último proceso de la columna sale del bucle, llama a crearUltimaFila() y espera a los y subhijos con un bucle de wait(NULL) antes de hacer exit(0).

Llamadas al Sistema Utilizadas: fork(), getpid(), wait(), exit().

```c
void crearProcesos(int x, int y){
    int pidsColumna[x];          // pids de los x procesos de la columna
    pid_t pid;

    // Columna de x procesos
    for(int i = 0; i < x; i++){
        pid = fork();

        switch (pid){
            case -1:
                // ERROR
                perror("Error en fork");
                exit(1);
            case 0:
                // HIJO
                pidsColumna[i] = getpid(); // El hijo se apunta y sigue

                break;
            default:
                // PADRE
                wait(NULL); // Todos los padres esperan

                if(i == 0){ // Si es el superpadre vuelve al main para hacer el print.
                    return;
                }

                exit(0); // Y los padres intermedios sí terminan
        }
    }

    // El último proceso de la columna crea los hijos finales
    crearUltimaFila(x, y, pidsColumna);

    for(int i = 0; i < y; i++) // El último proceso de la columna espera a todos sus hijos
        wait(NULL);
    exit(0);
}
```

Función crearUltimaFila()

La ejecuta únicamente el último proceso de la columna vertical para crear en abanico a los y subhijos finales. Realiza un bucle de y iteraciones con fork() donde cada hijo creado llama a codigoHijosFinales() pasándole su índice i y el vector heredado pidsColumna[], y después muere con exit(0).

Llamadas al Sistema Utilizadas: fork(), exit().

```c
void crearUltimaFila(int x, int y, const int pidsColumna[]){
    pid_t pid;

    for(int i = 0; i < y; i++){
        pid = fork();
        switch(pid){
            case -1:
                // ERROR
                perror("Error en fork");
                exit(1);
            case 0:
                // HIJO
                codigoHijosFinales(x, i, pidsColumna);
                exit(0);
        }
    }
}
```

3.4.4. Procesos hijo, registro en memoria compartida e impresión (codigoHijosFinales())

Función codigoHijosFinales()

Define el comportamiento de cada uno de los y subhijos de la última fila. Primero guarda su propio PID en su posición i dentro del array de memoria compartida (hijosFinales[i] = getpid()) para que luego lo pueda leer el superpadre, y a continuación imprime su mensaje recorriendo con un bucle el array pidsColumna[] que ha heredado con los PIDs de sus x padres.

Llamadas al Sistema Utilizadas: getpid().

```
// Cada hijo final se añade al array y hace su print.
void codigoHijosFinales(int x, int i, const int pidsColumna[]){
    hijosFinales[i] = getpid();

    printf("Soy el subhijo %d, mis padres son: ", getpid());

    for(int j = 0; j < x; j++){
        printf("%d", pidsColumna[j]);

        if(j < x - 1){ // Si no es el último ponemos coma
            printf(", ");
        }
    }
    printf("\n"); // y el salto de línea
}
```